

0.22.1. Remonter : `deplace_y` et un pas négatif — de base, ailleurs

Le 0.22 en colonne 10, avec le B : descente et montée.

1. Ce qu'il faut comprendre

C'est le 0.22, avec une autre lettre, le B (`ALPHABET[1]`, la 2e lettre), et une autre place : la **colonne 10**, dans les deux `deplace_y`.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.22.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1619 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    y = deplace_y(10, y, ALPHABET[1], pas);    // descente : y vaut 5
    y = deplace_y(10, y, ALPHABET[1], -pas);   // montée : y vaut 0

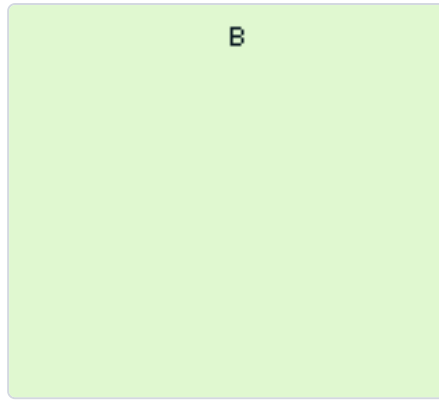
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

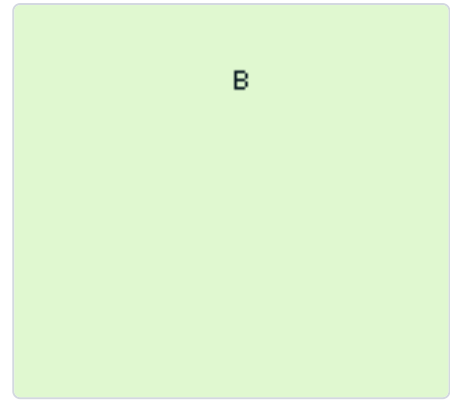
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



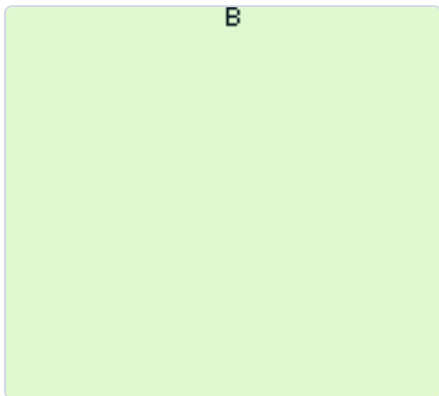
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui descend puis remonte, en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B passe par (10, 5)

✓ le B finit en (10, 0) (10,0)

Ensuite : **0.22.2. Remonter : `deplace_y` et un pas négatif — doublé, deux positions** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++